

Resolução de Exame — Paradigmas de Programação

Época Normal — Ano Letivo: 2025/2026

PARTE 1 – Perguntas Teóricas

Pergunta 1

As classes abstratas e as interfaces em Java apresentam distinções fundamentais na forma como gerem a herança, o estado interno e os construtores. Relativamente à herança, uma classe abstrata suporta apenas herança simples, o que significa que uma classe derivada pode estender no máximo uma única classe base, seja ela abstrata ou concreta. Em contrapartida, as interfaces suportam herança múltipla de tipo, permitindo que uma classe implemente várias interfaces em simultâneo.

No que diz respeito ao estado, as classes abstratas podem conter atributos de instância com qualquer modificador de acesso, permitindo armazenar dados mutáveis e partilhá-los com as subclasses. As interfaces não podem possuir variáveis de instância, definindo apenas constantes que são implicitamente públicas, estáticas e finais. Adicionalmente, as classes abstratas podem definir construtores que são invocados pelas subclasses por intermédio da instrução `super` para inicializar o estado herdado. As interfaces não contêm construtores porque não podem ser instanciadas e não contêm estado de instância para inicializar.

A nível de métodos, as classes abstratas admitem métodos concretos com corpo e métodos abstratos sem implementação. As interfaces permitem declarar métodos abstratos e, a partir de versões modernas do Java, métodos concretos com comportamento padrão (default), métodos estáticos com implementação e métodos privados para reutilização de código interno. Na intenção de design, a classe abstrata modela uma relação de identidade correspondente a ser um elemento da mesma família, enquanto a interface estabelece um contrato de comportamento ou capacidade independente da hierarquia da classe.

É adequado optar por uma classe abstrata quando se pretende partilhar estado e código comum entre classes estreitamente relacionadas na mesma hierarquia, como por exemplo uma superclasse `Veiculo` que armazena a matrícula e o nível de combustível comum a todas as subclasses, restando a cada subclasse concretizar o cálculo de autonomia individual. É preferível utilizar uma interface quando se deseja definir um contrato de comportamento comum que pode

ser implementado por classes dispersas em diferentes hierarquias e sem ligação direta, como por exemplo uma interface `ExportavelPDF` que define a operação de exportação, a qual pode ser implementada tanto por um `Relatorio` como por uma `Fatura` de forma independente.

```
public abstract class Veiculo {
    private String matricula;
    private double nivelCombustivel;

    public Veiculo(String matricula) {
        this.matricula = matricula;
        this.nivelCombustivel = 100.0;
    }

    public String getMatricula() {
        return matricula;
    }

    public void abastecer(double quantidade) {
        this.nivelCombustivel += quantidade;
    }

    public abstract double calcularAutonomia();
}

public class Carro extends Veiculo {
    private double consumoMedio;

    public Carro(String matricula, double consumoMedio) {
        super(matricula);
        this.consumoMedio = consumoMedio;
    }

    @Override
    public double calcularAutonomia() {
        return (100.0 / consumoMedio) * 50;
    }
}

public interface ExportavelPDF {
    byte[] exportarParaPDF();
}

public class Relatorio implements ExportavelPDF {
    private String conteudo;
```

```
@Override
public byte[] exportarParaPDF() {
    return conteudo.getBytes();
}

}

public class Fatura implements ExportavelPDF {
    private double valor;

    @Override
    public byte[] exportarParaPDF() {
        return ("Fatura no valor de " + valor).getBytes();
    }
}
```

Pergunta 2

Em Java, a passagem de argumentos para os métodos é efetuada exclusivamente por valor. Isto significa que, aquando da invocação de um método, é realizada uma cópia do valor armazenado na variável original e essa cópia é atribuída ao parâmetro local do método. Para os tipos primitivos, o valor copiado e transferido para a pilha de execução é o próprio dado numérico ou lógico. Consequentemente, qualquer modificação efetuada sobre a variável local no corpo do método afeta unicamente a sua cópia, deixando o valor original da variável chamadora inalterado.

No caso dos tipos de referência, que correspondem aos objetos e arrays, o valor armazenado na variável é o endereço de memória que aponta para o local na Heap onde o objeto reside. Assim, a passagem por valor copia este endereço de memória para o parâmetro do método. Dado que ambas as variáveis passam a conter o mesmo endereço, elas referenciam o mesmo objeto físico na Heap. Por este motivo, qualquer alteração ao estado interno do objeto efetuada dentro do método reflete-se e é visível externamente. No entanto, se o parâmetro local for reatribuído a uma nova instância utilizando o operador de instanciação, a cópia local do endereço é alterada para apontar para a nova área de memória, mantendo-se a referência original a apontar de forma inalterada para o objeto inicial. Existe o equívoco frequente de que os objetos são passados por referência, o que é contrariado pelo facto de que a reatribuição de um objeto dentro de um método não altera a referência da variável do método chamador.

```
public class TestePassagem {  
    public static void alterarPrimitivo(int numero) {  
        numero = 999;  
    }  
  
    public static void modificarEstadoObjeto(int[] array) {  
        array[0] = 50;  
    }  
  
    public static void reatribuirReferenciaObjeto(int[] array) {  
        array = new int[]{100, 200, 300};  
        array[0] = 999;  
    }  
  
    public static void main(String[] args) {  
        int a = 10;  
        alterarPrimitivo(a);  
        System.out.println("Primitivo 'a': " + a);  
  
        int[] meuArray = {1, 2, 3};  
        modificarEstadoObjeto(meuArray);  
        System.out.println("meuArray[0]: " + meuArray[0]);  
  
        reatribuirReferenciaObjeto(meuArray);  
        System.out.println("meuArray[0] apos reatribuicao: " + meuArray[0]);  
    }  
}
```

Pergunta 3

A conversão de tipos, ou casting, no contexto da herança e do polimorfismo, consiste em indicar explicitamente ao compilador como deve interpretar o tipo de uma referência de objeto na hierarquia. A conversão ascendente, conhecida por upcasting, consiste em converter uma referência de uma subclasse para uma superclasse. Esta operação é implícita, automática e sempre segura porque a subclasse estende e cumpre todas as características da superclasse. No entanto, o upcasting limita o acesso às operações específicas da subclasse, permitindo apenas a invocação dos métodos declarados na superclasse, embora a execução do método seja polimórfica e determinada em tempo de execução pela classe real do objeto.

A conversão descendente, conhecida por downcasting, consiste na conversão de uma referência de uma superclasse para uma subclasse. Esta operação é explícita e potencialmente perigosa. O compilador valida apenas se existe relação de parentesco na hierarquia, mas a compatibilidade real só é validada em tempo de execução pela máquina virtual Java. Se o objeto real na Heap não for compatível com o tipo de destino do cast, é lançada a exceção `ClassCastException`, a qual interrompe abruptamente a execução. Para mitigar este risco, deve validar-se previamente a compatibilidade do tipo através do operador `instanceof` antes de realizar o cast explícito.

```
public class Funcionario {  
    private String nome;  
  
    public Funcionario(String nome) {  
        this.nome = nome;  
    }  
  
    public void trabalhar() {  
        System.out.println(nome + " esta a trabalhar genericamente.");  
    }  
}  
  
public class Programador extends Funcionario {  
    public Programador(String nome) {  
        super(nome);  
    }  
  
    @Override  
    public void trabalhar() {  
        System.out.println("Escrevendo codigo em Java.");  
    }  
  
    public void programar() {  
        System.out.println("Efetuando commit no repositório.");  
    }  
}  
  
public class TesteCasting {  
    public static void main(String[] args) {  
        Funcionario func = new Programador("Alice");  
        func.trabalhar();  
  
        if (func instanceof Programador) {  
            Programador prog = (Programador) func;  
            prog.programar();  
        }  
  
        Funcionario funcReal = new Funcionario("Carlos");  
        try {  
            Programador progIncorreto = (Programador) funcReal;
```

```
        progIncorreto.programar();  
    } catch (ClassCastException e) {  
        System.out.println("Erro capturado: Nao e possivel converter Funcionario em Programador!")  
    }  
}  
}
```

Pergunta 4

A identidade de um objeto refere-se à sua localização física na memória Heap, sendo avaliada pelo operador de igualdade referencial que verifica se duas variáveis apontam exatamente para a mesma instância. A igualdade de um objeto diz respeito à equivalência semântica ou lógica do seu conteúdo, a qual é determinada pela implementação do método `equals` herdado de `Object`. Como a implementação padrão de `equals` na classe base `Object` se limita a efetuar uma comparação de identidade com o operador de igualdade, torna-se necessário sobrepor este método nas classes de negócio para definir critérios de igualdade lógica personalizados baseados em atributos.

O método `toString` tem a função de fornecer uma representação legível em formato de texto contendo o estado do objeto, sendo invocado implicitamente em concatenações de cadeias de caracteres ou na escrita para a consola. Para estruturar corretamente o método `equals`, deve verificar-se a igualdade referencial imediata como otimização de desempenho, de seguida validar se a referência comparada é nula, depois averiguar a correspondência exata de classes recorrendo ao método `getClass` para evitar incompatibilidades de herança, e por fim efetuar o cast seguro da referência para comparar os campos identificativos chave.


```
import java.util.Objects;

public class Estudante {
    private String numeroEstudante;
    private String nome;

    public Estudante(String numeroEstudante, String nome) {
        this.numeroEstudante = numeroEstudante;
        this.nome = nome;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (this.getClass() != obj.getClass()) {
            return false;
        }
        Estudante outro = (Estudante) obj;
        return Objects.equals(this.numeroEstudante, outro.numeroEstudante);
    }

    @Override
    public int hashCode() {
        return Objects.hash(numeroEstudante);
    }

    @Override
    public String toString() {
        return "Estudante [Nº: " + numeroEstudante + " | Nome: " + nome + "]";
    }
}
```

PARTE 2 – Programação em Java

Pergunta 1a

```
import java.util.Objects;

public class RefrigeratedVehicleImpl implements RefrigeratedVehicle {
    private final String code;
    private final ItemType supplyType;
    private final double maxCapacity;
    private final double maxKilometers;
    private VehicleState state;

    public RefrigeratedVehicleImpl(String code, ItemType supplyType, double maxCapacity, double maxKi
        if (code == null) {
            throw new IllegalArgumentException("O codigo do veiculo nao pode ser nulo.");
        }
        this.code = code;
        this.supplyType = supplyType;
        this.maxCapacity = maxCapacity;
        this.maxKilometers = maxKilometers;
        this.state = VehicleState.ENABLED;
    }

    @Override
    public String getCode() {
        return this.code;
    }

    @Override
    public ItemType getSupplyType() {
        return this.supplyType;
    }

    @Override
    public double getMaxCapacity() {
        return this.maxCapacity;
    }

    @Override
```

```
public double getMaxKilometers() {
    return this.maxKilometers;
}

public VehicleState getState() {
    return this.state;
}

public void setState(VehicleState state) {
    if (state == null) {
        throw new IllegalArgumentException("O estado do veiculo nao pode ser nulo.");
    }
    this.state = state;
}

@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (!(obj instanceof RefrigeratedVehicle)) {
        return false;
    }
    RefrigeratedVehicle other = (RefrigeratedVehicle) obj;
    return this.code.equals(other.getCode());
}

@Override
public int hashCode() {
    return Objects.hash(this.code);
}

@Override
public String toString() {
    return "Veiculo Refrigerado [Codigo: " + code + " | Tipo: " + supplyType + " | Estado: " + st
}
}
```

Pergunta 1b

```
public class TestRefrigeratedVehicle {  
    public static void main(String[] args) {  
        RefrigeratedVehicleImpl v1 = new RefrigeratedVehicleImpl("V-001", ItemType.PERISHABLE_FOOD, 1  
        RefrigeratedVehicleImpl v2 = new RefrigeratedVehicleImpl("V-001", ItemType.MEDICINE, 800.0, 3  
        RefrigeratedVehicleImpl v3 = new RefrigeratedVehicleImpl("V-002", ItemType.PERISHABLE_FOOD, 1  
  
        System.out.println("Codigo v1: " + v1.getCode());  
        System.out.println("Tipo carga v1: " + v1.getSupplyType());  
        System.out.println("Capacidade v1: " + v1.getMaxCapacity());  
        System.out.println("Distancia v1: " + v1.getMaxKilometers());  
  
        System.out.println("Estado inicial v1: " + v1.getState());  
        v1.setState(VehicleState.DISABLED);  
        System.out.println("Estado alterado v1: " + v1.getState());  
        v1.setState(VehicleState.ENABLED);  
  
        System.out.println("Igualdade v1 com v1: " + v1.equals(v1));  
        System.out.println("Igualdade v1 com v2 (mesmo codigo): " + v1.equals(v2));  
        System.out.println("Igualdade v1 com v3 (codigos diferentes): " + v1.equals(v3));  
        System.out.println("Igualdade v1 com nulo: " + v1.equals(null));  
        System.out.println("Igualdade v1 com outro tipo: " + v1.equals("Texto"));  
  
        System.out.println("Hash v1: " + v1.hashCode());  
        System.out.println("Hash v2: " + v2.hashCode());  
        System.out.println("Hash v3: " + v3.hashCode());  
  
        System.out.println("Representacao textual v1: " + v1.toString());  
    }  
}
```

Pergunta 2a

```
public class StrategyImpl implements Strategy {

    private boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox) {

        if (vehicle == null || aidbox == null) {
            return false;
        }
        Container[] containers = aidbox.getContainers();
        if (containers == null) {
            return false;
        }
        for (int i = 0; i < containers.length; i++) {
            Container current = containers[i];
            if (current == null) {
                continue;
            }
            if (current.getType() == vehicle.getSupplyType()) {
                Measurement last = current.getLastMeasurement();
                if (last != null) {
                    if (last.getValue() > (current.getCapacity() * 0.8)) {
                        return true;
                    }
                }
            }
        }
        return false;
    }

    private boolean addAidBoxToRoute(Route route, AidBox aidbox, RouteValidator validator) {
        if (route == null || aidbox == null || validator == null) {
            return false;
        }
        if (validator.validate(route, aidbox)) {
            try {
                route.addAidBox(aidbox);
                return true;
            } catch (RouteException e) {
                return false;
            }
        }
    }
}
```

```
        return false;
    }

    @Override
    public Route[] generate(IInstitution inst, RouteValidator validator) {
        return new Route[0];
    }
}
```

Pergunta 2b

```
public class StrategyImpl implements Strategy {

    private boolean hasCollectableContainer(Vehicle vehicle, AidBox aidbox) {

        if (vehicle == null || aidbox == null) {
            return false;
        }
        Container[] containers = aidbox.getContainers();
        if (containers == null) {
            return false;
        }
        for (int i = 0; i < containers.length; i++) {
            Container current = containers[i];
            if (current == null) {
                continue;
            }
            if (current.getType() == vehicle.getSupplyType()) {
                Measurement last = current.getLastMeasurement();
                if (last != null) {
                    if (last.getValue() > (current.getCapacity() * 0.8)) {
                        return true;
                    }
                }
            }
        }
        return false;
    }

    private boolean addAidBoxToRoute(Route route, AidBox aidbox, RouteValidator validator) {
        if (route == null || aidbox == null || validator == null) {
            return false;
        }
        if (validator.validate(route, aidbox)) {
            try {
                route.addAidBox(aidbox);
                return true;
            } catch (RouteException e) {
                return false;
            }
        }
    }
}
```

```
        return false;
    }

    @Override
    public Route[] generate(IInstitution inst, RouteValidator validator) {
        if (inst == null || validator == null) {
            return new Route[0];
        }
        Vehicle[] vehicles = inst.getVehicles();
        AidBox[] aidBoxes = inst.getAidBoxes();
        if (vehicles == null || aidBoxes == null || vehicles.length == 0 || aidBoxes.length == 0) {
            return new Route[0];
        }
        Route[] tempRoutes = new Route[vehicles.length];
        int routeCount = 0;
        for (int i = 0; i < vehicles.length; i++) {
            Vehicle currentVehicle = vehicles[i];
            if (currentVehicle == null) {
                continue;
            }
            Route currentRoute = new RouteImpl(currentVehicle);
            boolean routeHasAidBoxes = false;
            for (int j = 0; j < aidBoxes.length; j++) {
                AidBox currentAidBox = aidBoxes[j];
                if (currentAidBox == null) {
                    continue;
                }
                if (hasCollectableContainer(currentVehicle, currentAidBox)) {
                    if (addAidBoxToRoute(currentRoute, currentAidBox, validator)) {
                        routeHasAidBoxes = true;
                    }
                }
            }
            if (routeHasAidBoxes) {
                tempRoutes[routeCount] = currentRoute;
                routeCount++;
            }
        }
        Route[] finalRoutes = new Route[routeCount];
        for (int i = 0; i < routeCount; i++) {
            finalRoutes[i] = tempRoutes[i];
        }
    }
}
```



```
    }  
    return finalRoutes;  
  }  
}
```